

Punteros

Modularidad

Compilación

Antonio Espín Herranz

Contenidos

- Repaso de punteros
- Aritmética de punteros: desplazamientos, alineamiento y aliasing
- Punteros a funciones – callbacks, tablas de salto, máquinas de estados.
- Punteros void – uso genérico, conversión segura.
- Restricciones de aliasing y restrict – optimización y comportamiento indefinido.

Repaso de punteros

Punteros, conceptos, uso y aritmética

- Un puntero es una variable cuyo contenido es la dirección de una variable de cualquier tipo.
- Declaración:
 - tipo *nombre_puntero;
 - int *p; // Apunta a variables de tipo entero.
 - Un puntero solo puede apuntar a variables que sean de su propio tipo.
- Operadores:
 - * Devuelve o accede al contenido del puntero.
 - & Devuelve la dirección de memoria de una variable.

Punteros, conceptos, uso

- Ejemplo:
int numero = 6;
int *p;
p = №
*p = 10; // Se modifica el contenido de numero a partir de su dirección.
- Los punteros los podemos inicializar a NULL. Ejemplo p = NULL;
- En C, las variables puntero tienen que apuntar realmente a variables que sean del mismo tipo que el puntero.

Operadores para punteros

- Relacionales: `==`, `>`, `>=`, `<`, `<=`, `!=`
 - Podemos comparar las direcciones de memoria o el contenido de las direcciones.
 - `int x = 3, y = 5, *p, *q; p = &x; q = &y;`
 - `if (p == q) // Comparando las direcciones.`
 - `if (p > q) // Las direcciones de memoria.`
 - `if (*p != *q) // El contenido, 3 != 5.`

Operadores e impresión de punteros

- De incremento: ++ --
Podemos modificar el contenido de la variable:
`(*p)++;` // Forzamos a acceder al contenido.
`*(p++);` // Apuntará a la siguiente dirección de su tipo base.
`*(p + 1)` // Idem del anterior, pero sin modificar p.
En caso de ser un entero se le sumara 2 bytes.
- Con los punteros NO podemos:
 - Sumar dos punteros, multiplicar o dividir.
 - Pero si se pueden restar.
- Impresión:
`int n = 0, *p;`
`printf("n = %d, p = %p", n, p);`

Matrices: conceptos y manipulación

- El nombre de un array representa un puntero al primer elemento:
- En el caso de los punteros se pueden modificar, en el caso de los arrays no se pueden modificar.
- Manipulación:
`char str[100], *cad = NULL;`
`cad = str; // También: cad = &str[0];`

Matrices: conceptos y manipulación

// Impresión del contenido:

```
printf("%s %s", str, cad);
```

// Impresión de las direcciones:

```
printf("%p %p", str, cad);
```

// Distintas formas de almacenar una 'a' en el 3º carácter:

a) `str[2] = 'a';`

b) `cad += 2; *cad = 'a';` // No es muy usado, modifica el valor de cad.

c) `*(cad+2)='a';` // También vale: `*(str+2)='a';`

d) `cad[2] = 'a';`

Matrices de varias dimensiones

- Podemos tener arrays de varias dimensiones.
- Definición: `<tipo> nombre [nFilas][nCols];`
- Definición con inicialización:
 - `int tabla[2][3] = {51, 52, 53, 54, 55, 56};`
 - `int tabla[2][3] = { {51, 52, 53}, {54, 55, 56} };`
- Acceso a los elementos:
 - `tabla[0][0] = 99; printf(“%d”, tabla[1][0]);`
- Recorrer todos los elementos:
 - Utilizaremos un bucle **for** por cada dimensión. Máximo 256 dim.

Matrices como punteros y punteros a matrices

- El nombre del array es un puntero.
- Arrays de punteros: `int *p[10];` // Tenemos 10 punteros a enteros.
- `int *ptr1[];` // Array de punteros a int.
- Punteros a arrays: `tipo_elem_array (*puntero)[ ];`
 - `int (*ptr2)[];` // Puntero a un array de elementos int.
 - `int *(*ptr3)[];` // Puntero a un array de punteros a int.

Paso de matrices como parámetros

- Cuando pasamos un array como parámetro a una función tenemos que tener en cuenta que la función receptora no conoce el número de elementos del array.

// Lo podemos recibir como un puntero, pero no sabemos la longitud.

int sumaDeEnteros(int *); // Prototipo.

// Se puede pasar como un array y además indicar el número de elementos en otro parámetro.

int sumaDeEnteros(int [], int);

// Llamada:

int numeros[5] = {2,3,4,5,6};

sumaDeEnteros(numeros); // Se le pasa el nombre.

sumaDeEnteros(numeros, 5); // Se le pasa el nombre y número de elementos.

Los punteros como parámetros

- Es una forma de pasar los parámetros por referencia y poder modificar el contenido de las variables.
- Si queremos modificar una variable en un procedimiento tendremos que mandar la dirección de dicha variable.
- Si queremos modificar la dirección de un puntero, tendremos que mandar ese puntero por referencia, es decir, recibir un puntero a puntero.

Los punteros como parámetros

Paso de variables por referencia:

```
void intercambiar(int *a, int *b){  
    int aux;  
    aux=*a; *a = *b; *b = aux;  
}
```

```
void main(void){  
    int x = 3, y = 5;  
    intercambiar(&x, &y);  
}
```

Paso de punteros por referencia:

```
void intercambiar(int **a, int **b){  
    int *aux;  
    aux=*a; *a = *b; *b = aux;  
}
```

```
void main(void){  
    int x = 3, y = 5, *px, *py;  
  
    px = &x; py = &y;  
    intercambiar(&px, &py);  
}
```

Punteros a punteros

- Trabajaremos con un * mas.
- Definición: `<tipo> **<nombre_puntero>;`
- Ejemplos:
 - `char **p = NULL; // Se trata de un puntero a punteros a carácter.`
 - `char *nombres[] = { "tomas", "ana"};`
 - `p = nombres;`
 - `printf("%s, %c", *p, **p);`
 - **Salida → tomas, t**

Punteros Avanzados

Aritmética de punteros

- La aritmética de punteros en C está **escalada por el tamaño del tipo apuntado**.
- Esto significa que:
 - $p + 1$ **no avanza 1 byte**, sino `sizeof(*p)` bytes.
 - $p + n$ avanza $n * \text{sizeof}(*p)$ bytes.
- Los saltos en las direcciones se condicionan al tamaño del tipo:
 - 1 char $\rightarrow$ 1 byte, 1 int $\rightarrow$ 4 bytes, etc.

Ejemplo

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
int main(void) {
```

```
    int32_t v[4] = {10, 20, 30, 40};
```

```
    int32_t *p = v;
```

```
    printf("(p + 2) = %d\n", *(p + 2)); // 30
```

```
    printf("bytes entre p y p+1 = %td\n",
```

```
        (char*)(p+1) - (char*)p);    // 4 bytes en x86_64
```

```
}
```

Un puntero suele ocupar 8 bytes:
`sizeof(int*) == sizeof(char*)`

En Linux x86_64:

- `sizeof(int32_t) = 4`
- `sizeof(int64_t) = 8`
- `sizeof(double) = 8`

Desplazamientos (pointer offsets)

- El desplazamiento es avanzar con el puntero:
 - `uint8_t *p = buffer;` // acceso byte a byte
 - `int32_t *q = (int32_t*)buffer;` // acceso por palabras de 4 bytes
- `p += 1;` // avanza 1 byte
- `q += 1;` // avanza 4 bytes
- No es lo mismo: `p + 1` o `q + 1`, los anteriores modifican el inicio de la dirección del puntero ...

Alineamiento

- El alineamiento es la propiedad que exige que ciertos tipos estén ubicados en direcciones que son múltiplos de su tamaño natural.
- **En Ubuntu/Lubuntu x86_64:**
 - Los accesos desalineados **están permitidos**.
 - Pero son **más lentos** (penalización de hardware).
 - En casos extremos pueden generar *warnings* o comportamientos inesperados si se mezclan con optimizaciones agresivas.
- **En Ubuntu ARM (Raspberry Pi):**
 - Un acceso desalineado puede causar **SIGBUS** (crash inmediato).

Ejemplo

- Ejemplo de acceso desalineado:
 - `char buf[8];`
 - `int32_t *p = (int32_t*)(buf + 1);` // dirección no alineada a 4 bytes
 - `printf("%d\n", *p);` // en ARM puede fallar
- Para evitar problemas en la reinterpretar memoria:
 - Utilizar **`memcpy`**
 - O acceder bytes a byte con **`uint_t *`**

Ejemplo 2ª parte

```
uint32_t u;  
float f;
```

```
// evita aliasing y problemas de alineamiento  
memcpy(&f, &u, sizeof(f));
```

Aliasing

- Dos punteros pueden apuntar a la misma dirección de memoria:

```
int x = 5;  
int *a = &x;  
int *b = &x; // alias  
*b = 10;  
printf("%d\n", *a); // 10
```

El aliasing afecta a cómo **gcc** optimiza el código, porque debe asumir que escribir a través de un puntero puede modificar lo que otro puntero lee.

Punteros a funciones

- Podemos tener punteros que apunten a la dirección en memoria donde se encuentra una función.
- La dirección de la función viene indicada por el nombre de la misma.
 - De la misma forma que el nombre del array representa un puntero al primer elemento, el nombre de la función representa un puntero al inicio de la misma.
- Tipo_de_retorno (*PunteroFuncion)(<lista parametros)
 - `int f(int);` // Define la función f.
 - `int (*pf)(int)` // Define puntero pf a función con argumento int.
 - `pf = f;` // Asigna la dirección de f a pf

Asignación de la función al puntero

- `PunteroFuncion = unaFuncion;`

- Ejemplo:

```
double calculo(int *v, unsigned n); // prototipo de la función.
```

```
double (*qf)(int *, unsigned); // Puntero a función.
```

```
int r[11] = { 3, 5, 6, 7, 1, 7, 3, 34, 5, 11, 44 };
```

```
double x;
```

```
qf = calculo;
```

```
x = qf(r, 11); // Llamada a la función a partir del puntero.
```

Callbacks

- Los callbacks son funciones pasadas como parámetros a otras funciones. En C se implementa con punteros a funciones.
- Los callbacks permiten desacoplar la lógica y permitir comportamientos configurables.
- Se utilizan para APIs genéricas (qsort, bsearch, librerías propias).
 - Y eventos

Ejemplo

```
typedef void (*callback_t)(int result);
```

```
void do_work(callback_t cb) {  
    int r = 42; // resultado de algún cálculo  
    cb(r);      // invocamos el callback  
}
```

El tipo del puntero a función debe coincidir
Exactamente con la firma de la función.

```
void on_done(int result) {  
    printf("Resultado: %d\n", result);  
}
```

```
int main(void) {  
    do_work(on_done); // pasamos la función como parámetro  
}
```

Con datos de usuario, el callback necesita datos del usuario:

Callback con contexto

```
typedef void (*callback_t)(int result, void *user_data);
```

```
void do_work(callback_t cb, void *user_data) {  
    int r = 42; // El valor 42, simula algún cálculo y r es el resultado.  
    cb(r, user_data);  
}
```

```
typedef struct {  
    const char *tag;  
} context_t;
```

```
void on_done(int result, void *user_data) {  
    context_t *ctx = user_data;  
    printf("[%s] Resultado: %d\n", ctx->tag, result);  
}
```

```
int main(void) {  
    context_t ctx = { .tag = "TASK1" };  
    do_work(on_done, &ctx);  
}
```

callback_t

Define un tipo para puntero a función
Que recibe como parámetro: int y void *
Y devuelve void

Callbacks en APIs del sistema

- Por ejemplo, qsort necesita un **callback** de comparación:

```
int cmp_int(const void *a, const void *b) {  
    int ia = *(const int*)a;  
    int ib = *(const int*)b;  
    return (ia > ib) - (ia < ib); // -1, 0, 1  
}
```

```
int main(void) {  
    int v[] = {4, 1, 3, 2};  
    qsort(v, 4, sizeof(int), cmp_int);  
}
```

El **callback** controla el criterio de ordenación

Errores Típicos

- **Firmas incompatibles:** declarar el puntero a función con una firma y pasarle una función con otra firma → comportamiento indefinido.
- **User data mal casteado:** hacer `context_t *ctx = user_data;` y que `user_data` no apunte realmente a `context_t`.

Máquinas de Estado

- Una máquina de estados modela el comportamiento de un sistema como:
 - un conjunto de **estados**,
 - un conjunto de **transiciones**,
 - y una lógica que decide el siguiente estado según el actual y los eventos.
- **En C, esto se implementa con:**
 - Enum para los estados,
 - Se suele añadir un último estado que representa el número de estados, pero no se utiliza, solo como contador. STATE_COUNT.
 - Punteros a función para los manejadores de estado, almacenados en un array e indexados con valores de la enum.
 - Bucles que iteran sobre la máquina.

Ejemplo

```
#include <stdio.h>
```

```
typedef enum {  
    STATE_IDLE,  
    STATE_RUNNING,  
    STATE_ERROR,  
    STATE_COUNT // = 3, número de estados  
} state_t;
```

```
typedef state_t (*state_handler_t)(void);
```

```
state_t handle_idle(void) {  
    puts("IDLE: pasando a RUNNING");  
    return STATE_RUNNING;  
}
```

```
state_t handle_running(void) {  
    puts("RUNNING: pasando a IDLE");  
    return STATE_IDLE;  
}
```

```
state_t handle_error(void) {  
    puts("ERROR: pasando a IDLE");  
    return STATE_IDLE;  
}
```

```
int main(void) {  
    state_handler_t handlers[STATE_COUNT] = {  
        handle_idle,  
        handle_running,  
        handle_error  
    };  
  
    state_t st = STATE_IDLE;  
  
    for (int i = 0; i < 5; ++i) {  
        st = handlers[st](); // ejecuta lógica del estado y obtiene el  
                             siguiente  
    }  
}
```

Máquinas de estado con eventos

- Normalmente en las implementaciones mas reales, el estado puede recibir eventos que condicionen la transacción al nuevo estado.

- Los estados se definen dentro de otra enum:

```
typedef enum {  
    EVENT_START,  
    EVENT_STOP,  
    EVENT_FAIL  
} event_t;
```

Ejemplo

```
typedef enum {
    EVENT_START,
    EVENT_STOP,
    EVENT_FAIL
} event_t;

typedef struct {
    state_t state;
} fsm_t;

state_t idle_on_event(event_t ev) {
    switch (ev) {
        case EVENT_START: return STATE_RUNNING;
        default:          return STATE_IDLE;
    }
}

state_t running_on_event(event_t ev) {
    switch (ev) {
        case EVENT_STOP: return STATE_IDLE;
        case EVENT_FAIL: return STATE_ERROR;
        default:          return STATE_RUNNING;
    }
}
```

```
state_t error_on_event(event_t ev) {
    switch (ev) {
        case EVENT_START: return STATE_RUNNING;
        default:          return STATE_ERROR;
    }
}

// Array de estados de tamaño STATE_COUNT
state_t (*fsm_handlers[STATE_COUNT])(event_t) = {
    idle_on_event,
    running_on_event,
    error_on_event
};

void fsm_dispatch(fsm_t *fsm, event_t ev) {
    fsm->state = fsm_handlers[fsm->state](ev);
}
```

La máquina tiene un **estado actual** (fsm->state).
Cada evento se procesa por el handler correspondiente al estado actual.
El handler decide el **nuevo estado**.

Implementación de máquinas de estado finitas

- Una **enum** para los **estados**:
 - `typedef enum { STATE_IDLE, STATE_RUNNING, STATE_ERROR, STATE_COUNT};`
 - El último valor almacena el número de posibles estados.
- Una **función** por **estado**:
 - `state_t handle_idle(void);`
 - `state_t handle_running(void);`
 - `state_t handle_error(void);`

Implementación de máquinas de estado finitas II

- Una **tabla de punteros a función**:

```
state_handler_t handlers[STATE_COUNT] = {  
    handle_idle,  
    handle_running,  
    handle_error  
};
```

- Un **bucle** que ejecuta la FSM (Finite State Machine):

```
state_t st = STATE_IDLE; // Se inicializa en el primer estado.  
  
while (1) {  
    st = handlers[st]();  
}
```

Implementación de **FSM** con **eventos**

- Enum con los **estados**:

```
typedef enum {  
    STATE_IDLE,  
    STATE_RUNNING,  
    STATE_ERROR,  
    STATE_COUNT  
} state_t;
```

- Enum con los **eventos**:

```
typedef enum {  
    EVENT_START,  
    EVENT_STOP,  
    EVENT_FAIL,  
    EVENT_COUNT  
} event_t;
```

Implementación de FSM con eventos II

- **Estructura de contexto:**

- Contiene el **estado actual** y cualquier información adicional necesaria para procesar eventos (buffers, sockets, flags, etc.).

```
typedef struct {  
    state_t state; // El estado actual  
    int fd;          // ejemplo: socket asociado  
    void *userdata; // opcional  
} fsm_t;
```

Implementación de FSM con eventos III

- **Tipo de handler de estado:**

- Cada estado se maneja con una función que recibe un **evento** y devuelve el siguiente estado.
 - `typedef state_t (*state_handler_t)(fsm_t *fsm, event_t ev);`

- **Funciones manejadoras de cada estado:**

- `state_t idle_on_event(fsm_t *fsm, event_t ev);`
- `state_t running_on_event(fsm_t *fsm, event_t ev);`
- `state_t error_on_event(fsm_t *fsm, event_t ev);`

Implementación de FSM con eventos IV

- Tabla de los **handlers**:

```
state_handler_t handlers[STATE_COUNT] = {  
    idle_on_event,  
    running_on_event,  
    error_on_event  
};
```

- **Función de despacho de eventos:**

- Recibe un evento, consulta el estado actual y ejecuta el handler adecuado:

```
void fsm_dispatch(fsm_t *fsm, event_t ev) {  
    fsm->state = handlers[fsm->state](fsm, ev);  
}
```

Implementación de FSM con eventos V

- **Bucle principal de la FSM:**

- Procesa eventos provenientes de sockets, timers, señales o lógica interna:

```
fsm_t fsm = { .state = STATE_IDLE }; // Primer estado (inactivo)
```

```
while (fsm.state != STATE_ERROR) {  
    event_t ev = obtener_evento(); // lectura de socket, timer, etc.  
    fsm_dispatch(&fsm, ev);  
}
```

Ejemplos de prototipos

- **Comparativa de prototipos de funciones vs punteros a función:**
- `int *fp(void);` → Una función que devuelve un puntero a entero y no tiene parámetros.
- `int (*fp)(void);` → Puntero a una función que devuelve un entero y no tiene parámetros.
- `int **fp(void);` → Función que no tiene parámetros y que devuelve un puntero a puntero a entero.
- `int (*fp(void))(int);` → `fp` es una función que no tiene parámetros y que devuelve un puntero a una función que devuelve un entero y recibe un entero.
- `int *(*fp)(void);` → puntero a una función que devuelve un puntero a un entero y no tiene parámetros.
- `int (*( *pf)(void))(int)` → `pf` es un puntero a una función que no tiene parámetros y que devuelve un puntero a una función, que devuelve un entero y recibe un entero.

Punteros void

- Los punteros void son el **tipo de puntero genérico** en C. Representan una dirección de memoria sin información sobre el tipo de dato que contiene.
- **Se utilizan en:**
 - funciones del sistema (pthread_create, qsort, read, write, etc.),
 - APIs genéricas,
 - buffers de red,
 - estructuras de contexto,
 - librerías de propósito general.
 - ***Son fundamentales para escribir código flexible y desacoplado.***

Punteros void

- Un puntero **void**:
 - puede apuntar a cualquier tipo,
 - puede convertirse a cualquier otro puntero sin cast explícito,
 - no puede dereferenciarse directamente,
 - no permite aritmética de punteros según el estándar.

Ejemplo

```
int x = 10;
```

```
void *p = &x; // válido
```

```
*p = 20; // error: void no tiene tamaño
```

```
*(int*)p = 20; // correcto
```

Punteros void

- Conversión segura desde **void ***
 - Antes de acceder a memoria hay que convertirlo a un tipo concreto:

```
void print_int(void *ptr) {  
    int *p = (int*)ptr;  
    printf("%d\n", *p);  
}
```

- **void * → int *** es una conversión válida,
 - El compilador ya sabe el tamaño del tipo final.

Punteros void

- Conversión segura hacia void *
 - C permite convertir cualquier puntero a void * **sin cast explícito**
 - `int x = 5;`
 - `void *p = &x; // conversión implícita`
- Esto es útil para APIs genéricas:
 - callbacks,
 - funciones de hilos,
 - funciones de ordenación,
 - estructuras de contexto.

Ejemplo

- En las funciones que se pasan a los hilos, el prototipo recibe y devuelve punteros void.
- Nos permite pasar o devolver estructuras de cualquier tipo:

```
void *thread_main(void *arg) {  
    // arg es un void*  
}
```

Aritmética en void *

- En C estándar, siendo p: **void *p**;
 - **p+1** es un error, void no tiene tamaño.
- En **GNU C**:
 - Esto si funciona:
 - `void *p = buffer;`
 - `p = p + 1;` // avanza 1 byte (extensión GNU), pero NO ES PORTABLE
- La forma correcta con **char***
 - `void *next = (char*)p + 1;` // ***estándar, seguro y portable***

Recorrer bytes

- Cuando necesitamos recorrer memoria sin saber el tipo, debemos utilizar: `unsigned char*`

```
void dump(void *buf, size_t n) {  
    unsigned char *p = buf; // permitido por el estándar  
    for (size_t i = 0; i < n; ++i)  
        printf("%02X ", p[i]);  
}
```

Punteros void en callbacks

```
typedef void (*callback_t)(void *userdata);  
  
void do_work(callback_t cb, void *userdata) {  
    cb(userdata);  
}
```

Ejemplo

```
typedef struct {
```

```
    int id;
```

```
    const char *name;
```

```
} context_t;
```

```
void print_ctx(void *userdata) {
```

```
    context_t *ctx = userdata;
```

```
    printf("ID=%d, name=%s\n", ctx->id, ctx->name);
```

```
}
```

Punteros void en máquinas de estado

- Cuando el contexto incluye datos específicos suele usar:
void *

```
typedef struct {  
    state_t state;  
    void *userdata; // datos específicos del estado  
} fsm_t;
```

Errores con void *

- Cast incorrecto:
 - `double *p = userdata; // pero userdata era un int`
- Vida útil incorrecta:
 - Pasar un puntero a una variable local que ya no existe
- Aritmética directa sobre void *
 - `p++; // portable solo en GNU C`
- Dereferenciar sin cast
 - `*p = 10; // void no tiene tamaño`

Ejemplo: vida útil incorrecta

```
#include <stdio.h>
```

```
void procesar(void *ptr) {  
    int *p = ptr; // asumimos que ptr apunta a un int válido  
    printf("Valor: %d\n", *p);  
}
```

```
void crear_puntero_invalido(void) {  
    int x = 42; // variable local en el stack  
    procesar(&x); // ERROR: x deja de existir al salir de la función  
}
```

Definiendo static int x

```
int main(void) {  
    crear_puntero_invalido(); // UB: el puntero apunta a memoria destruida  
}
```

Restricciones de aliasing / restrict

- Aliasing dos punteros distintos apuntan a la misma dirección de memoria.

```
int x = 5;  
int *p = &x;  
int *q = &x; // p y q aliasan  
*q = 10;  
printf("%d\n", *p); // 10
```

Restricciones de aliasing / restrict

- El problema se da cuando el compilador asume que dos punteros de tipos distintos no aliasan, pero nosotros hacemos que sí aliasen, ahí entra la regla de aliasing estricto:
 - **strict aliasing rule**
- El compilador puede asumir que **punteros de tipos no compatibles no apuntan al mismo objeto.**

Restricciones de aliasing / restrict

- Aliasing se da cuando dos punteros distintos apuntan a la misma dirección de memoria.

```
int x = 5;
```

```
int *p = &x;
```

```
int *q = &x; // p y q aliasan
```

```
*q = 10;
```

```
printf("%d\n", *p); // 10
```

- Si los tipos son iguales (int* y int*) no hay problema.
- ***El problema se da cuando el compilador asume que punteros de distinto tipo no aliasen y forzamos que si aliasen.***

strict aliasing rule

- **Regla:** el compilador puede asumir que punteros de tipos no compatibles no apuntan al mismo objeto.
 - Eso le permite optimizar agresivamente:
 - mantener valores en registros sin recargar de memoria,
 - reordenar lecturas y escrituras,
 - eliminar accesos redundantes.
 - Si violamos esa suposición, entramos en el **comportamiento indefinido (UB)**: el compilador está en su derecho de generar código que no hace lo que esperamos.

strict aliasing rule

- Casos donde se permite el aliasing:
 - El estándar permite que un objeto se acceda a través de otro tipo **solo** en casos como:
 - Acceso mediante tipo carácter (char, unsigned char, signed char),
 - Acceso mediante su tipo original (por ejemplo, void * → cast de vuelta al tipo correcto),
 - Acceso mediante una Union que contenga ese tipo (con matices).
- **Válido:**
 - int num = 10;
 - unsigned char *p = (unsigned char*)# // **permitido: char puede aliasar cualquier tipo**

Ejemplo: No válido

- Partimos de una función:

```
void no_valido(float *f, int *i) {  
    *f = 1.0f;  
    *i = 42; // si f e i apuntan a la misma dirección → UB  
}
```

- Llamamos de esta forma:

```
int main(void) {  
    int x;  
    no_valido((float*)&x, &x);  
}
```

float* / int* apuntan al mismo objeto x.
El compilador asume que no aliasan
Puede reordenar las escrituras.

Al optimizar:

- El valor de x puede no ser 42
- Generar código inconsistente
- El programa se comporta de forma aleatoria

restrict

- Promesa explícita de no-aliasing:
 - **restrict** es un cualificador de puntero que indica al compilador que el puntero (y los derivados de él) solo accederá a esa región de memoria.

- Ejemplo de uso:

```
void add_vec(size_t n, int * restrict dst, const int * restrict a, const int *  
restrict b){  
    for (size_t i = 0; i < n; ++i)  
        dst[i] = a[i] + b[i];  
}
```

dst, a y b **no se solapan**,
el compilador puede asumir que escribir en dst[i] no afecta a a[i] ni b[i],
puede vectorizar el bucle, usar registros, reordenar accesos, etc.
Si en realidad sí se solapan (por ejemplo, llamas con dst == a), estás en **UB**: comportamiento indefinido.

gcc en la optimización

- Con -O2 y -fstrict-aliasing (activado por defecto en muchos casos) en gcc:
- **gcc asume** que punteros de tipos distintos no aliasan,
- Usa esa información para:
 - eliminar recargas de memoria,
 - reordenar instrucciones,
 - vectorizar bucles.

Ejemplo

```
void f(int *a, int *b) {  
    *a = 1;  
    *b = 2;  
    printf("%d\n", *a);  
}
```

Si a y b no aliasan, el compilador puede generar algo como:
escribir 1 en $*a$,
escribir 2 en $*b$,
imprimir 1 sin recargar $*a$ de memoria
(porque sabe que $*b$ no afecta a $*a$).

Si en realidad $a == b$, el comportamiento puede ser distinto de lo que esperas.

Buenas prácticas

- Para evitar UB (comportamiento indefinido) en el compilador:
 - Utilizar memcpy
 - Usar unsigned char* para acceso byte a byte.
 - Diseñar APIs que no dependan de aliasing complejos.

Ejemplo

```
float u32_to_float(uint32_t u) {  
    float f;  
    memcpy(&f, &u, sizeof(f)); // seguro y portable  
    return f;  
}
```

```
void dump(void *buf, size_t n) {  
    unsigned char *p = buf; // permitido por el estándar  
    for (size_t i = 0; i < n; ++i)  
        printf("%02X ", p[i]);  
}
```

restrict

- **Usarlo cuando:**
 - Tenemos funciones de **procesamiento de arrays** (vectores, imágenes, buffers),
 - sabemos que los parámetros **no se solapan**,
 - queremos que el compilador pueda optimizar sin miedo.

```
void scale_add(size_t n, float * restrict dst, const float * restrict src, float alpha){  
    for (size_t i = 0; i < n; ++i)  
        dst[i] += alpha * src[i];  
}
```

Si llamamos siempre con dst y src apuntando a regiones distintas, ganas rendimiento sin sacrificar corrección.

Qualifiers y const-correctness

Punteros constantes y punteros a constantes

- Punteros constantes:
 - `<tipo de dato> *const <nombre puntero> = <dirección>`
 - `int x, int *const p1 = &x;`
 - `p1` es un puntero constante, pero `*p1` es una variable, se puede cambiar el contenido de `p1`, pero no podemos alterar `p1`, es decir, el puntero.
 - `char *const nombre = "luis";`
 - `*nombre = 'c';` // Esta modificando el 1º carácter.
 - `nombre = &Otra_cadena;` // ERROR.

Punteros constantes y punteros a constantes

- Punteros a constantes:
 - `const <tipo de dato> *nombre_puntero = dirección de constante.`
 - `const int x = 25;`
 - `const int y = 50;`
 - `const int *p1 = &x;`
 - p1 se define como un puntero a la constante x. los datos son constantes y no el puntero, p1 puede apuntar a otra constante.
 - `p1 = &y;`
 - Pero no podemos modificar el contenido de p1.
 - `*p1 = 15; // ERROR.`

Punteros constantes a constantes

- `const <tipo de dato> *const <nombre_puntero> = constante;`
`const int x = 25;`
`const int *const p1 = &x;`
- No podemos modificar ni `p1` ni el contenido de `p1`.
`p1 = &otra_variable; // ERROR.`
`*p1 = 56; // ERROR.`

const en funciones

- Lo más habitual un puntero como parámetro de sólo lectura:

```
void print_message(const char *msg)
{
    printf("%s\n", msg);
}
```

- Nos daría error si dentro de la función hacemos:
 - `msg[0] = 'H';`

volatile

- Acceso a hardware, variables compartidas y optimización.
- El modificador indica que la variable puede cambiar en cualquier momento por causas externas al flujo normal del programa.
- El compilador puede realizar ciertas optimizaciones que pueden hacer que no se detecten cambios en una variable si se modifica fuera del ámbito del programa.

volatile

```
int flag = 0;  
while (flag == 0) { ... }
```

Con volatile:

- el hardware modifica el registro,
- el programa no tiene control sobre ello,
- el compilador debe leer memoria en cada iteración.

- Con este código el compilador puede optimizar con:

```
while (1){ ... }
```

- Para registros hardware:

```
volatile uint32_t *status = (volatile uint32_t *)0x40000000;  
while ((*status & 0x01) == 0) { ... }
```

static

- **static**: palabra clave que cambia el comportamiento de una variable o función dependiendo de dónde se utilice.

En C tiene **dos usos principales**:

1. Controlar la **vida útil** de una variable.
2. Controlar la **visibilidad** de funciones y variables entre ficheros.

Por eso suele decirse que static actúa sobre el **storage duration** y el **linkage**.

Variable local static

```
void contador(void)
{
    int x = 0;
    x++;
    printf("%d\n", x);
}
```

// Salida 1 1 1

```
void contador(void)
{
    static int x = 0;
    x++;
    printf("%d\n", x);
}
```

// Salida 1 2 3

Variables globales static

```
/* config.c */
```

```
int puerto = 8080;
```

- Esta variable seria accesible desde otros ficheros con extern.
- Si la hacemos **static**:
 - **static** int puerto = 8080;
 - Ningún otro módulo puede acceder a ella → **encapsulación**
 - Dar acceso a un recurso mediante una función pública del módulo

Funciones static

- Función visible desde otros módulos:
- `/* parser.c */`
 - `void parse_header(void) { ... }`
- Si añadimos **static**, la función es solo accesible desde el fichero **parse.c**
 - **static** `void parse_header(void){ ... }`

Resumen

- **static** modifica la duración o la visibilidad de variables y funciones.
- Una variable **local static** se inicializa una única vez y conserva su valor durante toda la ejecución.
- Una variable **global static** solo es visible dentro del fichero donde se declara.
- Una función static solo puede invocarse desde el fichero fuente donde está definida.
- Es una herramienta fundamental para encapsular módulos y evitar exposición innecesaria de símbolos.
- *En proyectos Linux es habitual declarar como static todas las funciones auxiliares y variables internas que no forman parte de la API pública del módulo.*

extern

- **static** sirve para ocultar elementos dentro de un fichero, **extern** sirve para hacer referencia a elementos definidos dentro de otro fichero.

src/

```
|— main.c
|— config.c
|— logger.c
|— network.c
```

Se compilan por separado y después el linker los une para generar el ejecutable final.

extern si definimos en config.c → int puerto = 8080 y la queremos Utilizar en main → extern int puerto.

En config.c se **define** y en main.c se **declara**.

Ejemplo con archivos.h

// config.h

```
#ifndef CONFIG_H
#define CONFIG_H
extern int puerto;
#endif
```

// config.c

```
#include "config.h"
int puerto = 8080;
```

// main.c

```
#include "config.h"
int main(void)
{
    printf("%d\n", puerto);
}
```

extern con funciones

- Con las funciones cuando escribimos:
 - `int sumar(int a, int b);`
- Ya estamos haciendo implícitamente:
 - `extern int sumar(int a, int b);`

Ejemplo

// calculadora.c

```
int sumar(int a, int b)
{
    return a + b;
}
```

// calculadora.h

```
int sumar(int a, int b);
```

// main.c

```
#include "calculadora.h"

int main(){
    printf("%d\n", sumar(2, 3));
}
```

Resumen

- **extern** declara variables o funciones definidas en otro fichero.
- No reserva memoria; únicamente informa al compilador de que el símbolo existe.
- Es el mecanismo habitual para compartir variables globales entre módulos.
- Las definiciones deben existir una única vez en el programa.
- `extern` y `static` son conceptos opuestos: `extern` expone símbolos y `static` los oculta al resto de módulos.
- En proyectos Linux se suele combinar `extern` en los archivos `.h` con una única definición real en un archivo `.c`.

Modularidad y diseño de APIs

Separación de los ficheros de definición y de implementación

- Los ficheros de definición se suelen definir tipos, variables (sin inicializar) y el prototipo de funciones.
- Este fichero tiene extensión **h**.
- En el prototipo de una función solo se indica la cabecera de la función.
 - `float suma(float, float);`
- En el fichero de implementación se desarrolla el código de todas las funciones. Tendrá extensión **C**.

Ámbito de las funciones definidas en un módulo

- Una función definida dentro de un módulo solo se puede usar dentro de ese módulo.
- Si queremos que esta función se pueda utilizar en otro módulo, tendremos que definirla como **extern**.
- En el caso de una variable ocurre lo mismo puedo tener una variable global a nivel de módulo o a nivel de todos los módulos, si la defino como **extern**.

Ejemplo

```
/* Funciones.h */  
#ifndef prueba  
    #define prueba  
    extern int suma(int, int);  
    extern int resta(int, int);  
#endif
```

```
/* Funciones.c */  
  
#include "funciones.h"  
  
extern int suma(int a, int b){  
    return(a + b);  
}  
  
extern int resta(int a, int b){  
    return(a - b);  
}
```

```
/* Principal.c */
```

```
#include <stdio.h>  
#include "funciones.h"
```

```
int main(){  
  
    int a = 10, b = 6;  
  
    printf("la suma es: %d", suma(a,b));  
    printf("la resta es: %d", resta(a,b));  
    return 0;  
}
```

Ámbitos

- Ámbitos de funciones y variables:
 - global
 - local
 - estático

local

- Las variables locales se declaran dentro de una función o bloque y solo existen mientras se ejecuta dicho bloque.
- Dentro de una función:
 - Solo son visibles dentro de la función.
 - Se crean al entrar en la función.
 - Se destruyen al salir.
 - Normalmente se almacenan en el stack.

global

- Las variables globales se declaran fuera de cualquier función y son accesibles desde todo el fichero.
 - Si no son static, también pueden compartirse entre módulos mediante `extern`.

Características:

- Existen durante toda la ejecución del programa.
- Son visibles desde cualquier función del fichero.
- Pueden **compartirse** entre archivos `.c` mediante **`extern`**.

static

static puede aplicarse tanto a variables como a funciones.

Variable local static (dentro de una función)

Características:

- Solo es visible dentro de la función.
- Se inicializa una única vez.
- Conserva su valor entre llamadas.
- Existe durante toda la vida del programa.

static

Variable global static

Características:

- Existe durante toda la ejecución.
- Solo es visible dentro del fichero donde se declara.
- No puede accederse mediante extern.

static

- **Función static**

Características:

- Solo puede invocarse desde el mismo fichero .c.
- Se utiliza para funciones auxiliares o privadas del módulo.

Resumen

Tipo	Visible desde	Vida útil
Local	La función o bloque donde se declara	Mientras se ejecuta el bloque
Global	Todo el fichero (y otros módulos con <code>extern</code>)	Toda la ejecución
Local <code>static</code>	Solo la función donde se declara	Toda la ejecución
Global <code>static</code>	Solo el fichero donde se declara	Toda la ejecución
Función <code>static</code>	Solo el fichero donde se define	Toda la ejecución del programa

Compilación Avanzada

Compilación condicional

- Son instrucciones del **preprocesador**.

- Sintaxis:

```
#if expresion_constante  
    sentencias;  
#endif
```

- Ejemplo:

```
#define uno 1  
#if uno  
    int multiplicar(int a, int b){  
        return (a*b);  
    }  
#endif
```

Compilación condicional

Con parte else:

```
#if expresion
    sentencias;
#else
    sentencias;
#endif
```

```
#define uno 0
#if uno
    int multiplicar(int a,int b)
#else
    int multiplicar(a,b)
        int a, b;
#endif
{
    return(a*b);
}
```

Compilación condicional

- Si el identificador está definido compila las sentencias:

```
#ifdef identificador  
    sentencias;  
#endif
```

- Si el identificador NO está definido compila las sentencias:

```
#ifndef identificador  
    sentencias;  
#endif
```

Ejemplo

- Uso muy común en la inclusión de ficheros de cabecera en varios ficheros fuente, evitamos duplicidades, cuando se compilan todos los ficheros.
- En este ejemplo tenemos un fichero tipos.h que lo van a usar dos fuentes de C.

Ejemplo

- **Tipos.h**

```
#ifndef TIPO
#define TIPO
typedef struct p {
    float x;
    float y;
} Tpunto;
#endif
```

- **Main.c**

```
#include "Tipos.h"

int main(){
    Tpunto p;
    .....
}
```

- **Aux.c**

```
#include "Tipos.h"

void calcular(Tpunto p, Tpunto q){
    ...
}
```

Compilación condicional

```
#if expr1  
    sentencias;
```

```
#elif expr2  
    sentencias;
```

```
#elif expr3  
    sentencias;
```

```
#endif
```

```
#define uno 1  
#define dos uno+uno  
#define tres dos+uno
```

```
#if uno==3  
    imprimir("hola");
```

```
#elif dos==2  
    imprimir("hola2");
```

```
#elif tres==3  
    imprimir("hola3");
```

```
#endif
```

#pragma

- Directiva del compilador.
- El estándar C permite que los compiladores definan sus propios pragmas.
- No es portable entre compiladores
- gcc/clang tienen sus propios pragmas
- Se usa para controlar comportamientos especiales del compilador

#pragma

- #pragma le dice al compilador: “**aplica una regla especial aquí**”.
- Ejemplos típicos:
 - desactivar warnings,
 - cambiar alineamiento de estructuras,
 - activar optimizaciones específicas,
 - marcar funciones para atributos especiales,
 - controlar el empaquetado de estructuras,
 - gestionar regiones críticas para análisis estático.

#pragma pack

- Control de alineamiento y empaquetado:

```
#pragma pack(push, 1)
struct header {
    uint8_t type;
    uint32_t length;
};
#pragma pack(pop)
```

- Esto fuerza que la estructura tenga **alineamiento 1 byte**, útil para:
 - protocolos binarios,
 - estructuras recibidas por red,
 - lectura de archivos binarios.

#pragma pack

- Problemas con el alineamiento:
- En Ubuntu/Lubuntu (gcc/clang), cada tipo tiene un **alineamiento natural**:
 - uint8_t → alineamiento 1
 - uint16_t → alineamiento 2
 - uint32_t → alineamiento 4
 - uint64_t → alineamiento 8
- Esto significa que el compilador intenta colocar cada campo en una dirección que sea múltiplo de su alineamiento.

Ejemplo

- Sin `#pragma pack`

```
struct header {  
    uint8_t type;    // 1 byte  
    uint32_t length; // 4 bytes  
};
```

El compilador:

- Detecta offset 1, por el campo type
- El compilador **inserta padding** para llegar a offset 4:
 - offset 0: type (1 byte)
 - offset 1: padding (3 bytes)
 - offset 4: length (4 bytes)

TOTAL: 8 bytes

- type ocupa 1 byte → offset 0
- length requiere alineamiento 4 → debe colocarse en una dirección múltiplo de 4

Ejemplo 2ª parte

- Fuerza a que **todos los campos** de la estructura tengan alineamiento **1 byte (con esto no inserta padding)**

```
#pragma pack(push, 1)
```

```
struct header {
```

```
    uint8_t type;
```

```
    uint32_t length;
```

```
};
```

```
#pragma pack(pop)
```

offset 0: type (1 byte)
offset 1: length (4 bytes)

TOTAL: 5 bytes

Problemas pack(1)

Accesos desalineados

En x86_64 (Ubuntu/Lubuntu):

- se permiten,
- pero son **más lentos**.

En ARM (Raspberry Pi con Ubuntu):

- pueden causar **SIGBUS**.

Pruebas

```
#include <stdio.h>
#include <stdint.h>
```

```
struct normal_header {
    uint8_t type;
    uint32_t length;
};
```

```
#pragma pack(push, 1)
struct packed_header {
    uint8_t type;
    uint32_t length;
};
#pragma pack(pop)
```

```
int main(void)
{
    printf("normal_header = %zu\n", sizeof(struct normal_header));
    printf("packed_header = %zu\n", sizeof(struct packed_header));
}
```

#pragma GCC diagnostic

- Para controlar warnings:

```
#pragma GCC diagnostic push
```

```
#pragma GCC diagnostic ignored "-Wunused-variable"
```

```
void f() {
```

```
    int x; // no genera warning
```

```
}
```

```
#pragma GCC diagnostic pop
```

- Permite:
 - desactivar warnings específicos,
 - reactivarlos después,
 - evitar que el código se llene de flags de compilación.

#pragma once

- Evitar múltiples inclusiones:
 - Se coloca al inicio de un .h y evita que el archivo se incluya varias veces.
- Equivalente a:

```
#ifndef HEADER_H
#define HEADER_H
...
#endif
```

Directivas del procesador

- Macros:
 - Se crean con `#define` y se puede eliminar con `#undef`
 - Pueden tener parámetros → ojo con los efectos colaterales.
- Includes:
 - Inclusión de librerías de compilador `<fichero.h>`
 - O de ficheros externos: `"fichero.h"`
 - Se puede añadir un path delante del nombre del fichero.

Macros

- Se aconseja utilizar funciones inline en vez de macros. Ya que las funciones chequean el tipo.
- Se utiliza la directiva **#define**.
- OJO con los efectos colaterales.

```
#define MENOR(x, y) ((x) < (y) ? (x) : (y))
```

```
// Con la llamada m = MENOR(a--, b--);
```

```
// El valor del menor se decrementa dos veces, por la sustitución de la macro.
```

- **Utilizar funciones inline**, que no tienen estos efectos.

Directivas del procesador

- **include:**
 - Añadir **#pragma once**
 - O añadir **guards**:

```
#ifndef PERSONA_H
#define PERSONA_H
struct persona {
    int edad;
};
#endif
```

Dependencias circulares

- En A.h hacemos include B.h
- En B.h hacemos include A.h
- En este caso, aunque haya **guards** nos dará problemas.
- Aquí añadir una declaración adelantada:
 - Por ejemplo: struct B; en lugar de include “B.h”